



Lab 10 – Worksheet

Name: Emaan Fatima, Abdul Ghani, Arsheel Abbas	ID: 11031, 11943, 11163	Section: T3
--	----------------------------	-------------

Task a. 10-bit counter

Code: Design module `h_counter` & testbench

Provide appropriately commented code for designed module, the code should contain meaningful variable naming. Modify the testbench provided in Fig 10.9(b) to verify the functionality of your designed `h_counter` module.

**Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.*

Code:

```
module h_counter(clk, h_count, trig_v);
    input clk;
    output reg [9:0] h_count;
    output trig_v;

    initial h_count = 0;

    assign trig_v = (h_count == 799) ? 1'b1 : 1'b0;

    always @(posedge clk) begin
        if (h_count < 799) begin
            h_count <= h_count + 1;
        end else begin
            h_count <= 0;
        end
    end
endmodule
```

TestBench:

```
module tb_h_counter();
    reg clk;
    wire [9:0] h_count;
    wire trig_v;

    h_counter uut(.clk(clk), .h_count(h_count), .trig_v(trig_v));

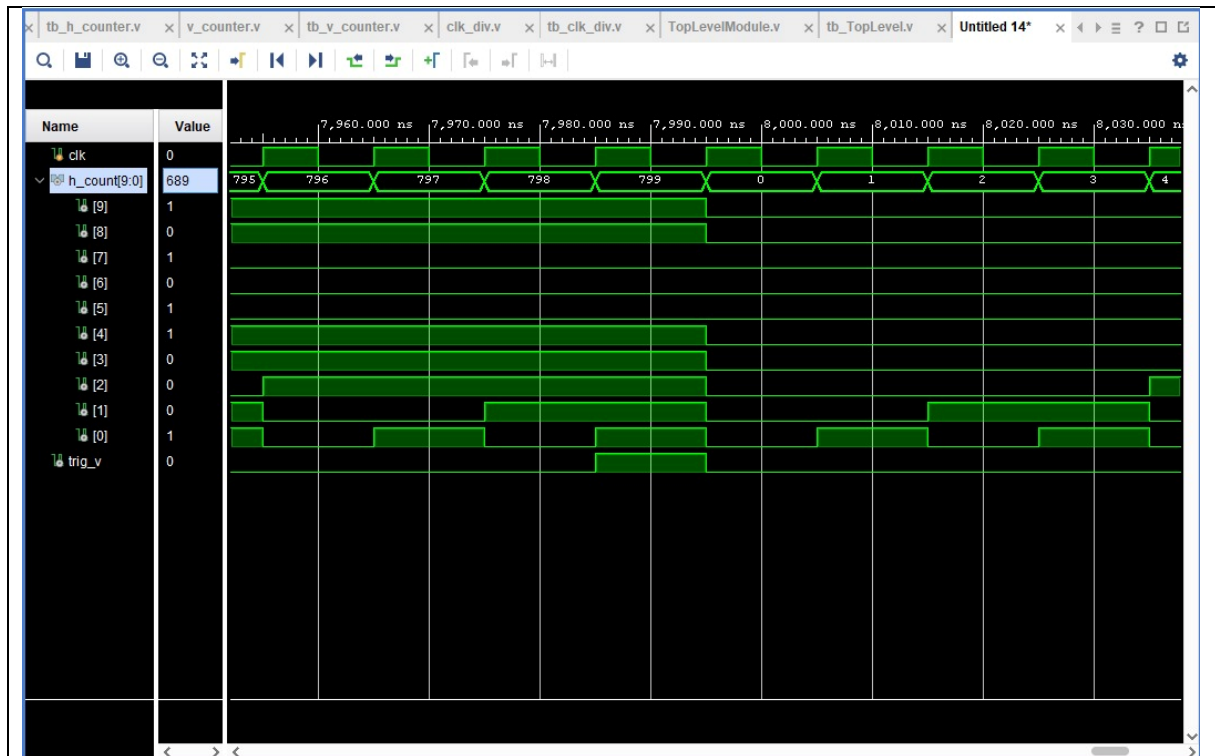
    initial clk = 1'b0;
```



```
always #5 clk = ~clk;  
  
initial begin  
    #8500 $finish;  
end  
endmodule
```

Results (Waveforms)

**Add snip of relevant signals' waveforms. Make sure the irrelevant area of the snip is cropped.*



Comments

**Observation/Comments on the obtained results/working of code.*

The code operates as a synchronous 10-bit counter that increments exactly once every 10ns, driven by the 100 MHz clock defined in the testbench. A key observation is the relationship between the `h_count` and the `trig_v` output; the counter is designed to count from 0 up to 799, at which point `trig_v` pulses high to signal a completed line. However, because the testbench is set to `$finish` at 200ns, the simulation only captures the first 20 clock cycles. This means while the incrementing logic can be verified, the simulation ends long before the counter reaches its maximum value of 799, so the rollover logic and the activation of the `trig_v` signal would not be observed without increasing the simulation runtime to at least 8,000ns.

Concept check:

Assuming that a time unit, in above mentioned testbench, is defined in nano seconds; what is the frequency at which the designed `h_counter` module would operate?

100 MHz

**Task b.****Code: Design module v_counter and its testbench**

Provide appropriately commented code for designed module and the testbench, the, code should contain meaningful variable naming.

Modify the testbench provided in **Error! Reference source not found.**(b) to verify the functionality of your designed v_counter module.

**Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.*

Code:

```
module v_counter(clk, enable_v, v_count);
    input clk;
    input enable_v;
    output reg [9:0] v_count;

    initial v_count = 0;

    always @(posedge clk) begin
        if (enable_v == 1'b1) begin
            if (v_count < 524)
                v_count <= v_count + 1;
            else
                v_count <= 0;
        end
    end
endmodule
```

TestBench:

```
module tb_v_counter();
    reg clk, enable_v;
    wire [9:0] v_count;

    v_counter uut(.clk(clk), .enable_v(enable_v), .v_count(v_count));

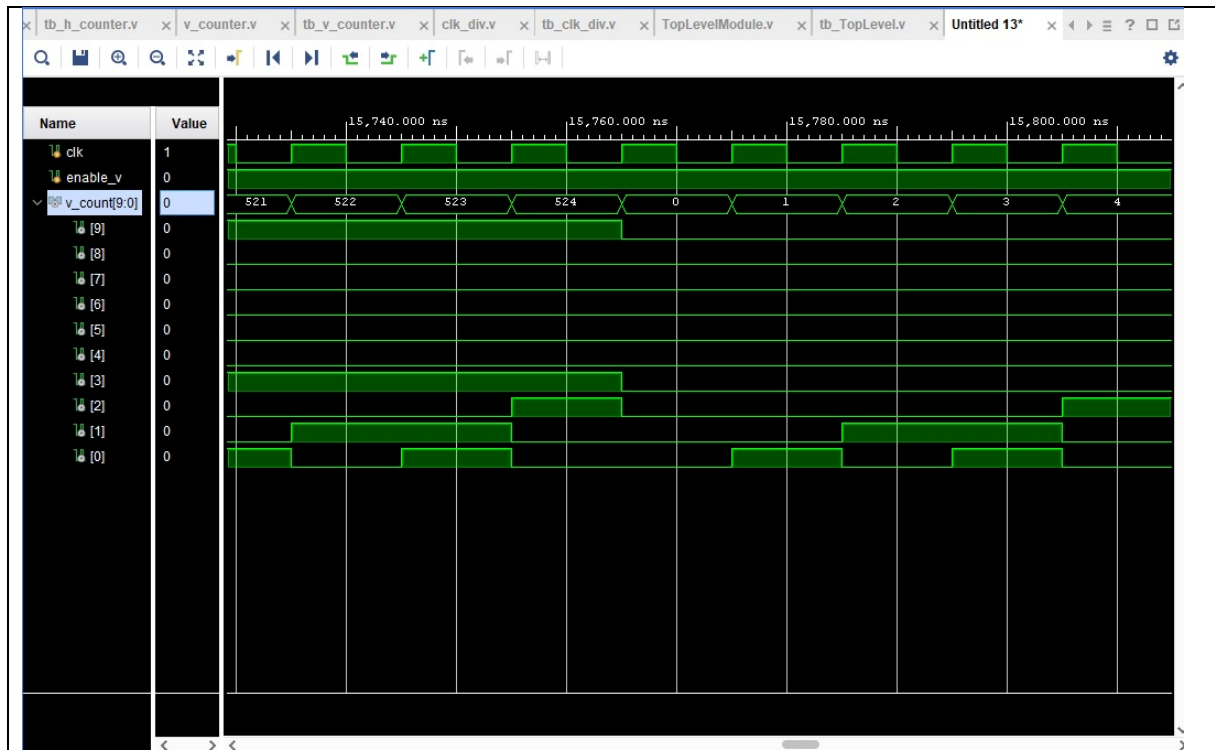
    initial clk = 1'b0;
    always #5 clk = ~clk;

    initial begin
        enable_v = 1'b0;
        #20 enable_v = 1'b1;
        #6000 $finish;
    end
endmodule
```



Results (Waveforms)

**Add snip of relevant signals' waveforms. Make sure the irrelevant area of the snip is cropped.*



Comments

**Observation/Comments on the obtained results/working of code.*

This module functions as a vertical counter that tracks the line position of a display, operating synchronously with the main clock but gated by the `enable_v` signal. The logic ensures that `v_count` only increments when `enable_v` is asserted, which in a standard VGA setup occurs once for every full horizontal line completed (799 cycles of the horizontal counter). The code effectively manages the vertical boundary by counting from 0 to 524; once it hits the maximum value of 524, it resets to 0 on the next enabled clock edge, resulting in a total of 525 vertical lines. This hierarchical design is efficient because it prevents the vertical counter from running at the high-speed pixel clock rate, instead updating only when specifically triggered by the horizontal stage's rollover signal.

Task c. Clock Divider

Before moving to the design module, calculate the `div_value` using formula provided in manual:

To get the 25MHz clock needed for the VGA output from the 100MHz Basys3 onboard clock, I used the formula from the manual:

$$\text{div_value} = \frac{100,000,000}{2 \times 25,000,000} - 1$$



Simplifying the fraction gives $100 / 50$, which is 2. Subtracting 1 results in a **div_value** of 1.

This makes sense because, as shown in the timing diagram, the counter only needs to go from 0 to 1 before toggling the output clock. By toggling every time the counter hits 1, the new clock signal stays high for two cycles and low for two cycles of the original clock, successfully dividing the frequency by 4 to hit that 25MHz target.

Code: Design module & testbench

Provide appropriately commented code for designed module & its testbench, code should contain meaningful variable naming.

**Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.*



Code:

```
module clk_div (clk, clk_d);
    parameter div_value = 1;
    input clk;
    output reg clk_d;
    reg count;

    initial begin
        clk_d = 0;
        count = 0;
    end

    always @(posedge clk) begin
        if (count == div_value)
            count <= 0;
        else
            count <= count + 1;
        end

    always @(posedge clk) begin
        if (count == div_value)
            clk_d <= ~clk_d;
        end
    end
endmodule
```

TestBench:

```
module tb_clk_div();
    reg clk;
    wire clk_d;

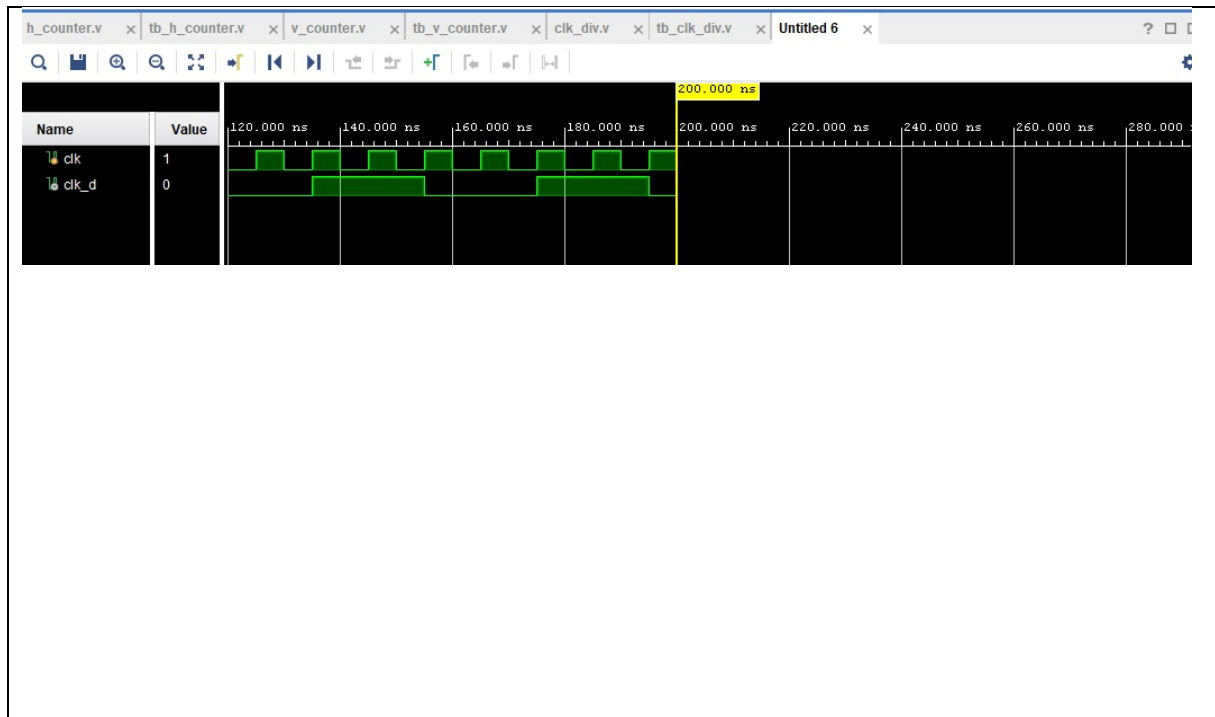
    clk_div uut(.clk(clk), .clk_d(clk_d));

    initial clk = 1'b0;
    always #5 clk = ~clk;

    initial #200 $finish;
endmodule
```

Results (Waveforms)

**Add snip of relevant signals' waveforms. Make sure the irrelevant area of the snip is cropped.*



Comments

**Observation/Comments on the obtained results/working of code.*

The code implements a basic clock divider that successfully reduces the 100MHz input clock to the 25MHz required for VGA. By using a `div_value` of 1, the `count` register toggles between 0 and 1 on every positive edge of the clock. Whenever the counter reaches its limit, the `clk_d` signal is flipped, creating an output clock that is high for two cycles and low for two cycles of the original signal. This effectively multiplies the clock period by four.

The testbench is set up correctly with a 10ns period (#5 toggle), which matches the Basys3 hardware clock. In the simulation, you would observe `clk_d` completing a full cycle every 40ns, confirming the 25MHz output. One thing to note is that the `count` and `clk_d` logic are split into two separate `always` blocks; while this works fine in simulation, it relies on the fact that they both trigger on the same clock edge to keep the toggle synchronized with the counter reset. Finally, since the testbench runs for 200ns, you'll be able to see exactly five full cycles of the new 25MHz clock before the simulation finishes.

Exercise

Show your Lab results to the RA and get it signed by the RA.

--



Calculate the div_value using the formula provided in manual:

Done Above

Code: Design module & testbench

Provide appropriately commented code for designed module & its testbench, code should contain meaningful variable naming.

*Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.

Code:

```
module tb_TopLevel();
    reg clk;
    wire [9:0] h_count, v_count;

    TopLevelModule uut(.clk(clk), .h_count(h_count), .v_count(v_count));

    initial clk = 1'b0;
    always #5 clk = ~clk;

    initial #50000000 $finish;
endmodule
```

TestBench:

```
module TopLevelModule(clk, h_count, v_count);
    input clk;
    output [9:0] h_count;
    output [9:0] v_count;

    wire clk_d;
    wire trig_v;

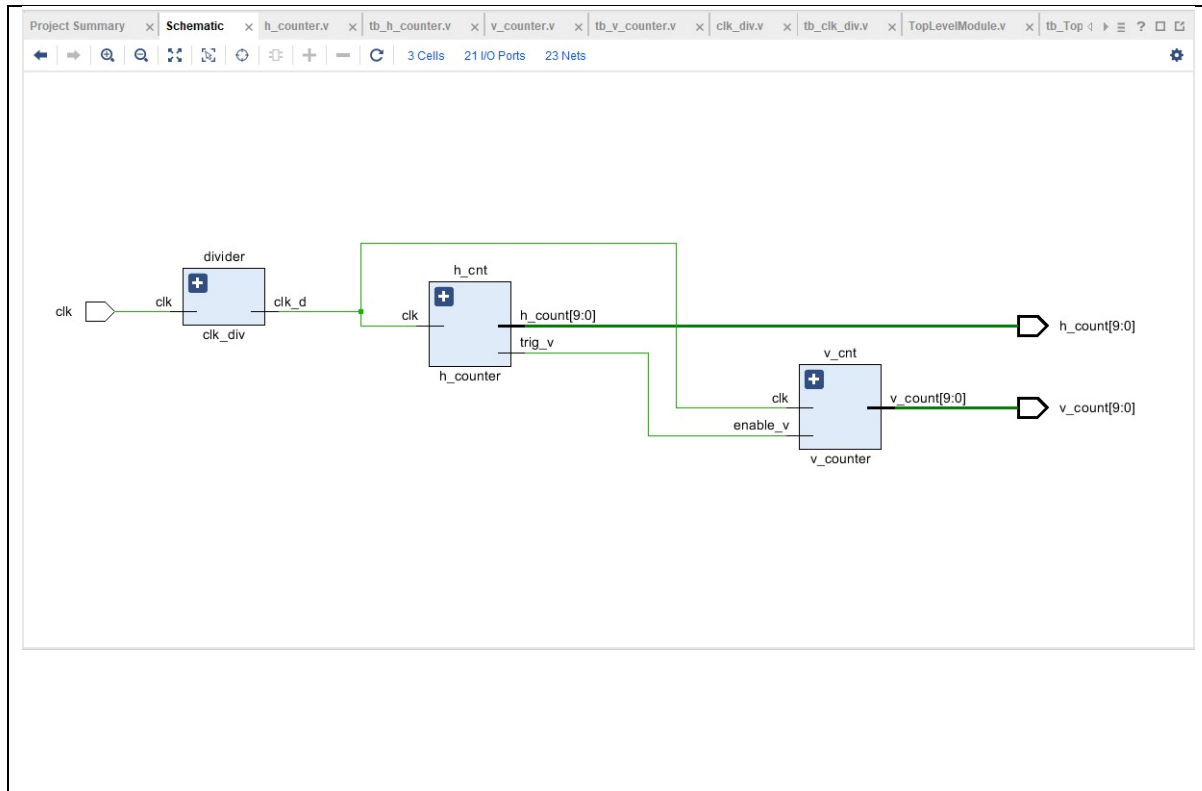
    // Set div_value to 9999 for 5KHz
    clk_div #(.div_value(9999)) divider (
        .clk(clk),
        .clk_d(clk_d)
    )
endmodule
```



```
);  
  
h_counter h_cnt (  
    .clk(clk_d),  
    .h_count(h_count),  
    .trig_v(trig_v)  
);  
  
v_counter v_cnt (  
    .clk(clk_d),  
    .enable_v(trig_v),  
    .v_count(v_count)  
);  
endmodule
```

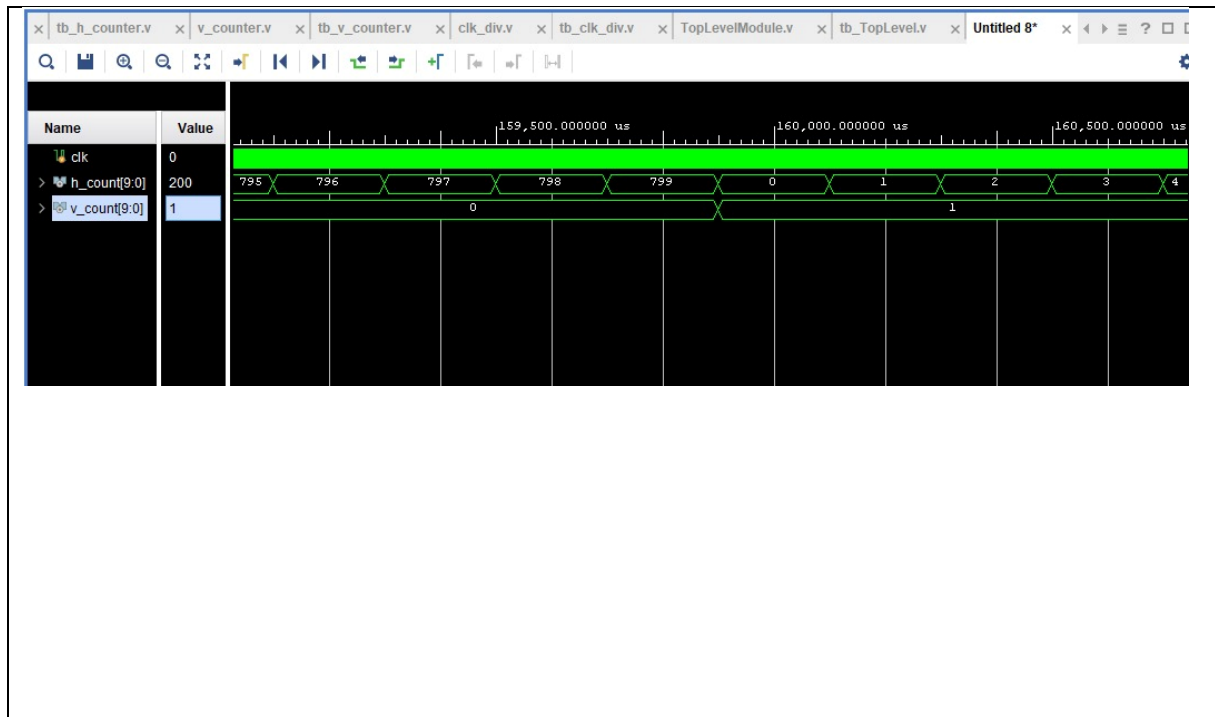


RTL Schematic:



Results (Waveforms)

**Add snip of relevant signals' waveforms. Make sure the irrelevant area of the snip is cropped.*



Comments

**Observation/Comments on the obtained results/working of code.*

This top-level module successfully integrates the clock divider, horizontal counter, and vertical counter into a single working hierarchy. By overriding the `div_value` to 9999, the 100MHz clock is scaled down to 5kHz, making the transitions slow enough to monitor easily. The design correctly uses the divided clock (`clk_d`) to drive both counters, while the `trig_v` signal from the horizontal stage acts as the enable for the vertical stage. With the testbench set to run for 50,000,000ns, there is finally enough time to observe the full cycle of the horizontal counter and verify that the vertical counter increments properly each time a line is completed.

Reflection:

This lab provided a practical look at building a synchronized digital system by moving from basic logic gates to a modular VGA timing controller. By calculating the clock division needed for the Basys3 board, I learned how to scale a high-frequency 100MHz signal down to a usable 25MHz rate for hardware interfacing. Developing the horizontal and vertical counters separately showed the power of modular design, specifically how a single trigger signal can coordinate two distinct hardware blocks to track 2D coordinates. Ultimately, integrating these into a top-level module demonstrated how structural Verilog connects individual logic units into a cohesive "orchestrator" capable of managing complex display timing.

**Assessment Rubric
Lab 10 Worksheet
Counter and Clock divider**

Name: Emaan Fatima, Abdul Ghani, Arsheel Abbas	Student ID: 11031, 11943, 11163
---	---------------------------------

Marks Distribution:

		LR2 Code	LR5 Results
In - Lab	Task a	20	10
	Task b	10	10
	Task c	-	-
Exercise		20	20
Total Marks: 100		/50	/40
CLO Mapping		CLO2	CLO2

Affective Domain Rubric		Points	CLO Mapped
AR 7	Viva	/10	CLO 4

CLO	Total Points	Points Obtained
2	90	
4	10	
Total	100	

For description of different levels of the mapped rubrics, please refer to the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.



Lab Evaluation Rubrics

#	Assessment Elements	Level 1: Unsatisfactory	Level 2: Developing	Level 3: Good	Level 4: Exemplary
LR2	Program/Code/ Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR7	Viva	Response shows a complete lack of understanding of the assigned / completed task	Response shows shallow understanding of the assigned task	Response shows substantial understanding of the assigned task	Response shows complete understanding of the completed task. The student is able to explain all the related concepts.